Nama : Arief Purnama Muharram
NIM : 23521013

Greedy example program

```
def greedy (items, matCost, keyFunction):
    itemsCopy = sorted (items, key = keyFunction, reverse = True)
    result = [ ]
    totalValue, totalCost = 0.0, 0.0
    for i in range(len(itemsCopy)):
        if (totalCost + itemsCopy[i].getCost) ≤ matCost :
            result.append (itemsCopy[i])
            totalCost += itemsCopy[i].getCost
            totalValue += itemsCopy[i].getValue
    return (result, totalValue)
```

Memo dynamic program example → use only w/ recursive

```
def fastFib (n, memo={ }):
    if n == 0 or n == 1 :
        return 1
    try :
        return memo[n]
    except KeyError :
        result = fastFib (n-1, memo) + fastFib (n-2, memo)
        memo[n] = result
        return result
```

Slice string

S[idx_start ; idx_end ; step]

>> S[0:4:1]

Sort

↳ Matole list

>> L.sort()

>> L.reverse()

Sorted

↳ Return sorted list array

>> sorted (L)

OOP

__ → make private attribute

↳ access using

>> _Cls__name (✓)

>> self._cls__name (✓)

```
class Car :
    def __init__ (self):
        super (Car, self).__init__()

class Toyota (Car):
    def __init__ (self):
        ~~super (Car)~~
        Car.__init__(self)
```

Assert

>> assert [kondisi benar], [pesan error]

>> assert int ("Lorem ipsum"), " bukan angka!") (✗)

Error catch

```
try :
    [Block of codes]
except :
    [Block of codes]
else :
    [Block of codes]
finally :
    [Block of codes]
```

SyntaxError
[NameError]
AttributeError
[TypeError]
[ValueError]
IOError

Override class

__str__(self) → "print" statement

__add__(self) → "add" operator

__eq__(self) → "equal" operator

Files

w, a, r

with open (<filename>, "w") as fh :

<expression> → fh.read()

fh.readline()

fh.readlines()

fh.writeline()

fh.writelines()

fh.close()

Recursive ~~steps~~

```
if n == 1 :
    return 1
else :
    return n * factorial (n-1)
```

```
def isPalindrome(s):
    def toChars(s):
        s = s.lower()
        ans = " "
        for c in s:
            if c in "....."
                ans = ans + c
        return ans

    def isPal(s):
        if len(s) # <= 1:
            return True
        else:
            return s[0] == s[-1] and isPal(s[1:-1])

    return isPal(toChars(s))
```

Main program:

```
>> if __name__ == "__main__":
>>     pass
```